

CIFAR-100 CNN Experiment: AMD Cloud / ROCm Results Walkthrough

1. Purpose of this section

This section documents the CIFAR-100 CNN experiments that were run in a Jupyter notebook using an AMD Cloud / ROCm environment. This is not the full main assignment report by itself. Instead, it is a results walkthrough used to compare the AMD Cloud run against a Google Colab Free environment.

The purpose was to test how a convolutional neural network performs on a smaller CIFAR-100 subset when trained with GPU acceleration in a ROCm-enabled PyTorch environment. The results help show how different model and training choices affected final test accuracy.

2. Environment

The notebook was run in Jupyter using PyTorch and torchvision. The environment confirmed that GPU acceleration was available. PyTorch reported `torch.cuda.is_available()` as `True`. In PyTorch, the device interface still uses the word `cuda`, but in this case the installed build was ROCm-enabled and was running on AMD GPU infrastructure.

The reported versions were:

PyTorch version: `2.9.0.dev20250821+rocm7.0.0.git125803b7`

torchvision version: `0.24.0+rocm7.0.0.git98f8b375`

GPU available: `True`

This confirms that the notebook was not using a basic CPU-only environment. It was running with GPU support through the ROCm PyTorch build.

```
[6]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import torchvision
import numpy as np
import matplotlib.pyplot as plt

print(f"PyTorch version: {torch.__version__}")
print(f"GPU available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")

PyTorch version: 2.9.0.dev20250821+rocm7.0.0.git125803b7
GPU available: True
GPU:
```

Figure 1. AMD Cloud / ROCm Jupyter environment showing the ROCm-enabled PyTorch version and confirming that GPU acceleration was available.

3. Dataset setup

The CIFAR-100 dataset was used for the experiment. Instead of training on all 100 classes, the notebook selected a smaller subset of 10 classes. This made the experiment more suitable for limited training time and easier to compare with a Google Colab Free run.

The selected classes in the first dataset setup were:

apple, aquarium_fish, baby, bear, beaver, bed, bee, beetle, bicycle, and bottle.

After filtering the dataset, the notebook used:

Training samples: **5,000**

Test samples: **1,000**

Image shape: **32 × 32 × 3**

The images were normalized so that pixel values were scaled between 0 and 1 before training.

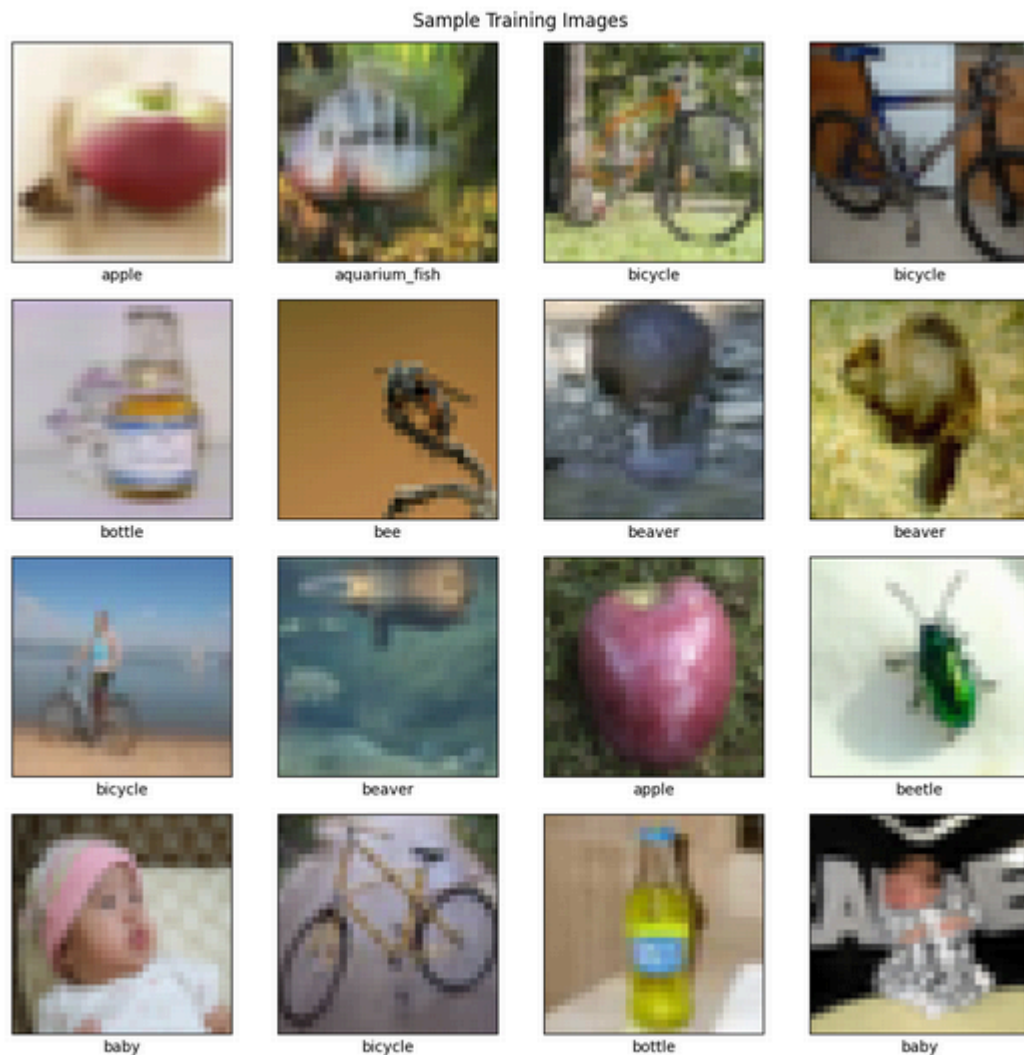


Figure 2. Example images from the selected 10-class CIFAR-100 training subset used in the baseline experiments.

4. Model and training approach

The experiments used convolutional neural networks trained from scratch. The models used convolutional layers to learn visual features from the images, max pooling layers to reduce feature map size, dropout to reduce overfitting, and dense layers for final classification.

The notebook tested several changes, including deeper convolutional blocks, a larger dense classifier, LeakyReLU activation, different optimizers, different batch sizes, and a separate animal-focused class subset.

The goal was not only to train one model, but to experiment with changes and observe how each change affected final test accuracy.

5. Experiment results

The uploaded notebook shows several final test accuracy values. The corrected results from the newest file are:

Experiment	Description	Final test accuracy
Baseline	Original CNN using ReLU, Adam optimizer, batch size 64	61.10%
Exp 1: CNN DeepDense	CNN with a larger dense classifier section	60.50%
Exp 2: CNN 4Block	Deeper four-block convolutional model	58.20%
Exp 3: LeakyReLU	CNN using LeakyReLU activation with Adam optimizer	71.90%
Exp 4: SGD Optimizer	LeakyReLU CNN using SGD with momentum	57.60%
Exp 5b: Batch Size 32	Adam optimizer with batch size 32	66.20%
Dataset 2: Animal-class subset	Separate 10-class animal subset using the best selected configuration	71.20%
Dataset 2: Final/baseline output	Final animal-class output shown at the end of the notebook	57.00%

The baseline model reached **61.10%** final test accuracy. The best overall result was **71.90%**, achieved by **Exp 3: LeakyReLU with Adam**. The SGD optimizer experiment reached **57.60%**, so it was not the best result. The animal-class subset also reached a strong **71.20%** in one run, while a later final output for that subset reached **57.00%**.

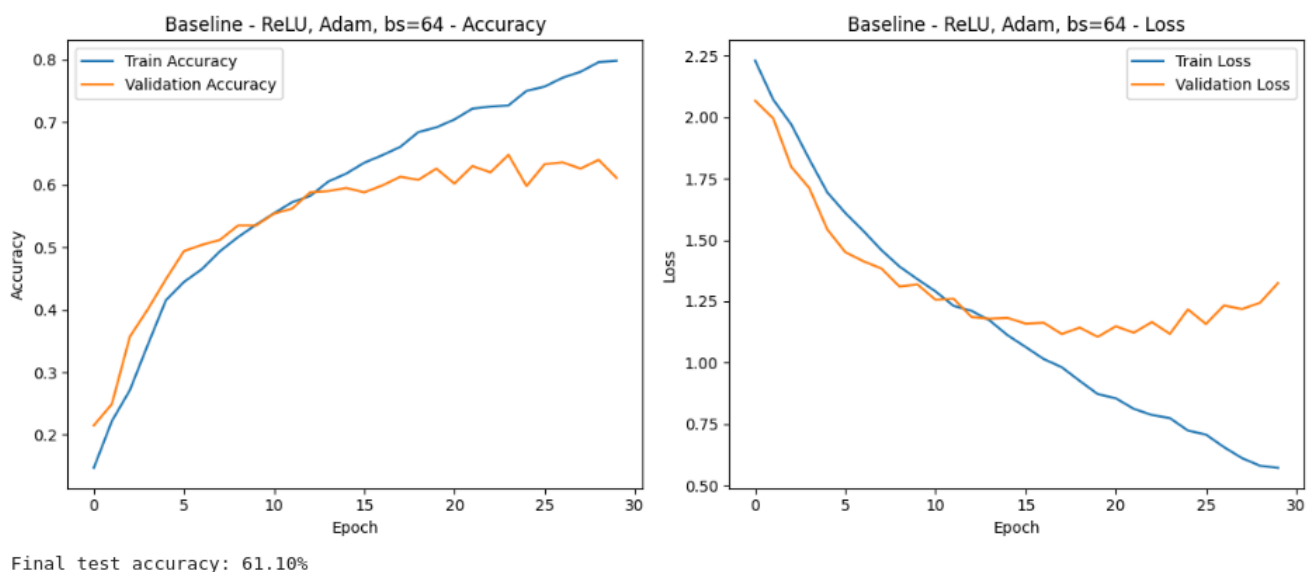


Figure 3. Final test accuracy for the baseline CNN experiment, using ReLU, Adam optimizer, and batch size 64.

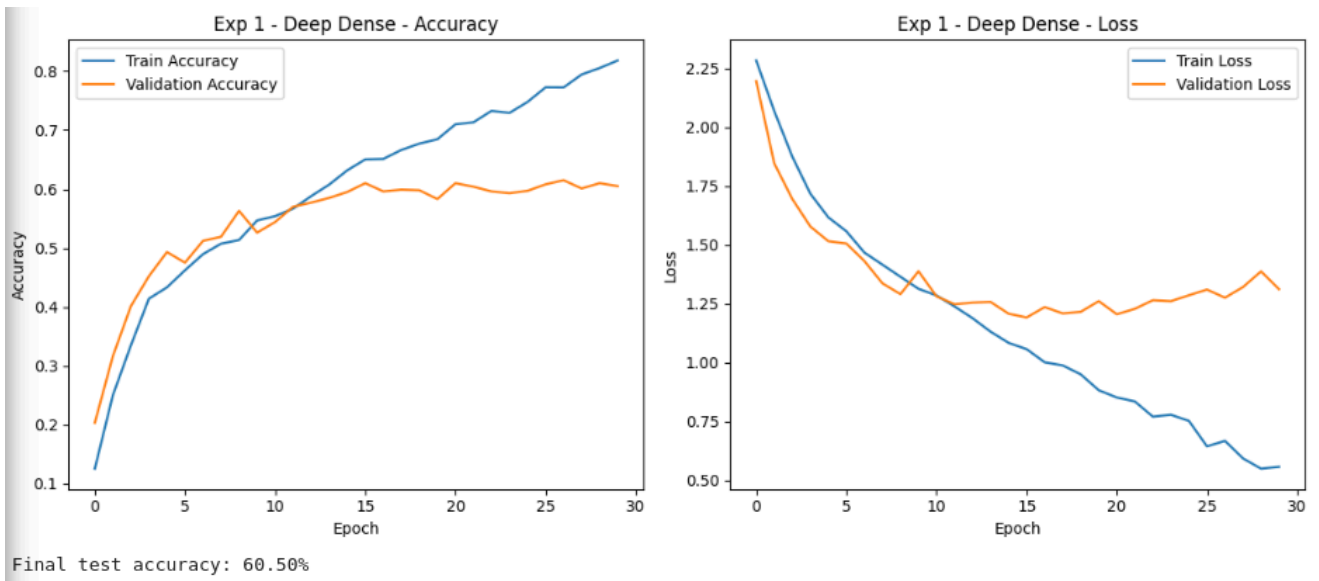


Figure 4. Final test accuracy for **Exp 1: Deep Dense**, reaching **60.50%**.

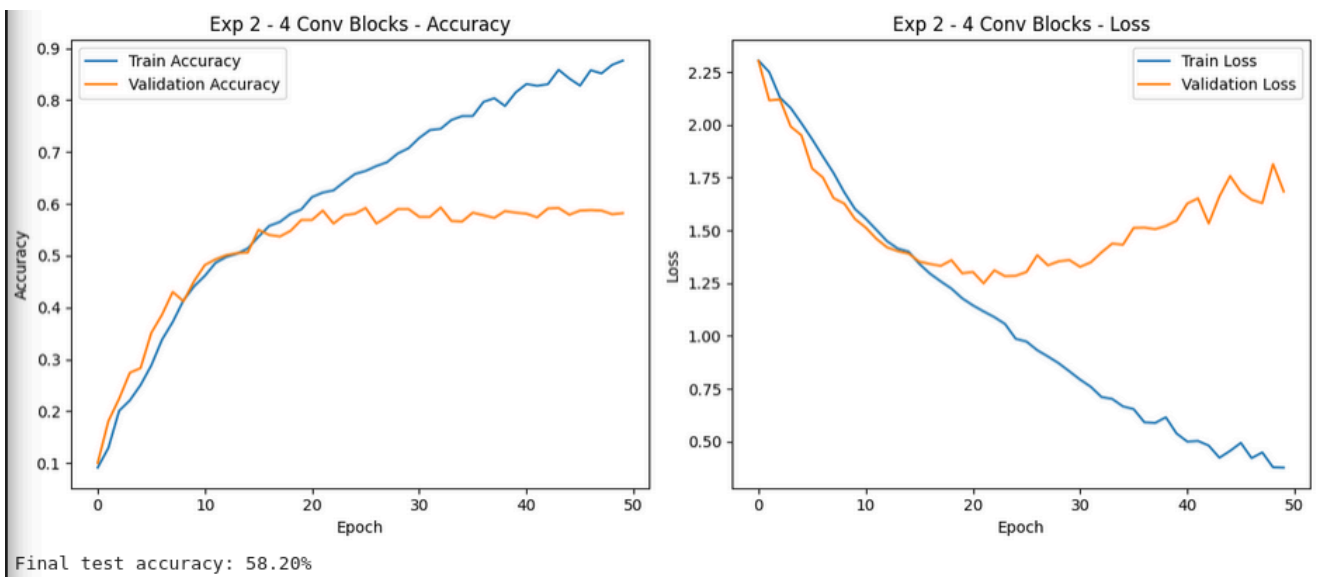


Figure 5. Final test accuracy for **Exp 2: 4 Conv Blocks**, reaching **58.20%**.

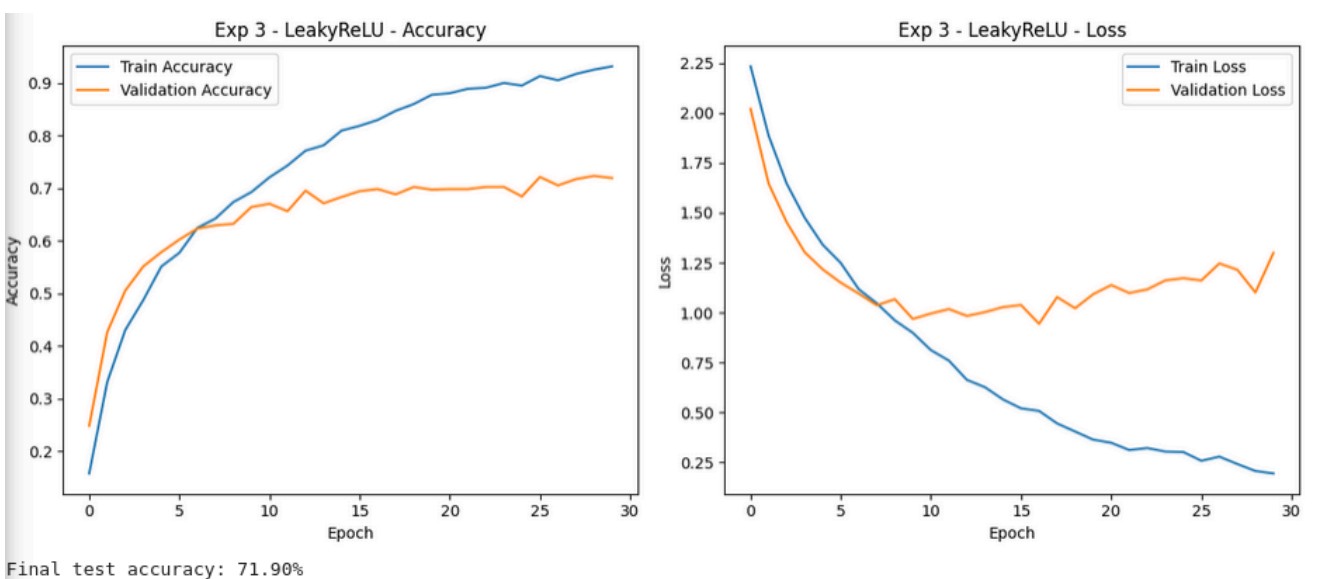


Figure 6. Best observed AMD ROCm result from Exp 3 using LeakyReLU with Adam, reaching **71.90%** final test accuracy.

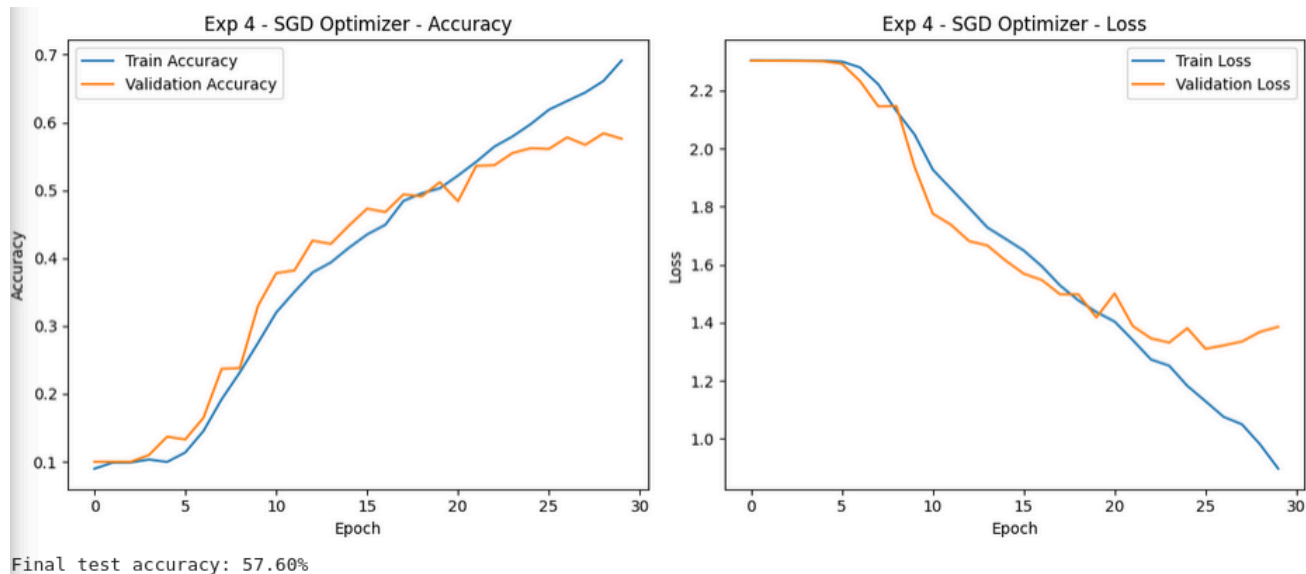


Figure 7. Final test accuracy for Exp 4 using the SGD optimizer with momentum, reaching **57.60%**.

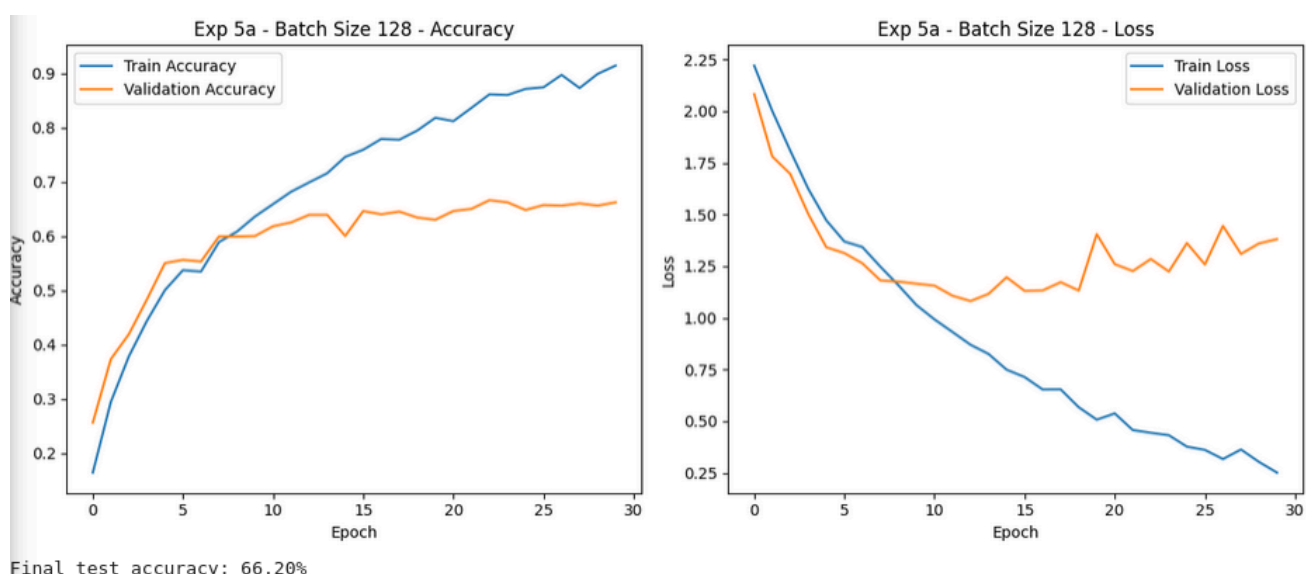


Figure 8. Final test accuracy for **Exp 5a: Batch Size 128**, reaching **66.20%**.

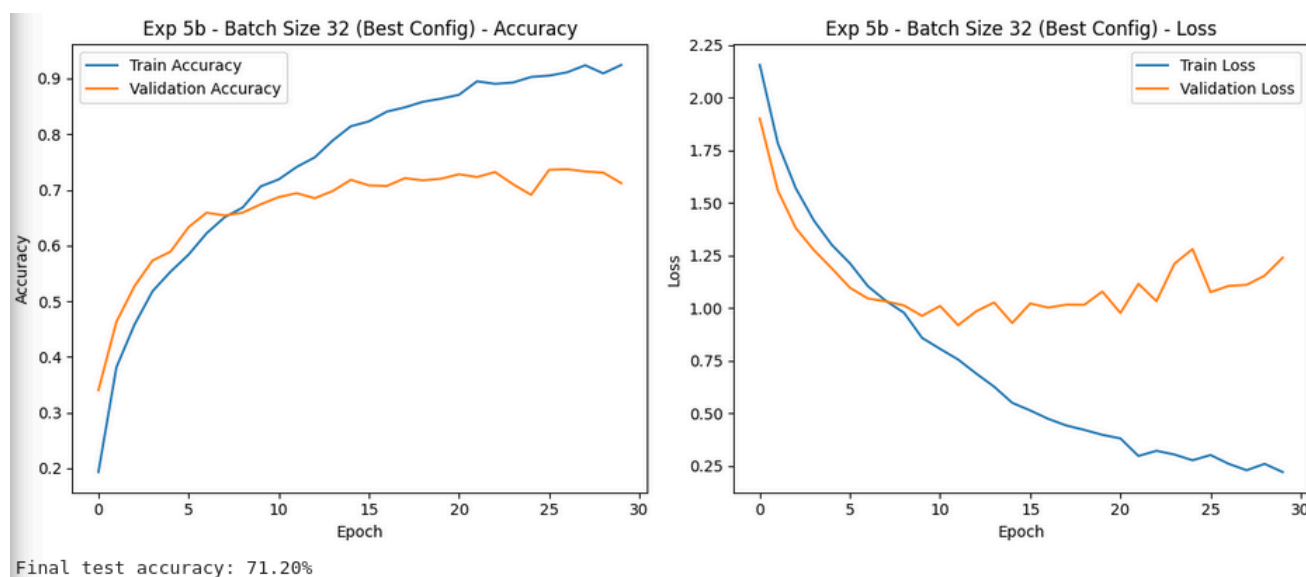


Figure 9. Final test accuracy for **Exp 5b: Batch Size 32**, reaching **71.20%**.



Figure 10. Accuracy and loss curves for **Dataset 2: Animal Classes, Best Config**, reaching **57.00%** final test accuracy and showing signs of overfitting.

6. Result interpretation

The results show that CNN performance changed noticeably depending on the model structure, activation function, optimizer, batch size, and selected classes.

The baseline model reached **61.10%** final test accuracy. Exp 1, the CNN DeepDense model, reached **60.50%**, and Exp 2, the CNN 4Block model, reached **58.20%**. These results show that simply increasing the dense classifier size or adding a deeper convolutional structure did not automatically improve performance over the baseline.

The best result was achieved by **Exp 3: LeakyReLU with Adam**, which reached **71.90%** final test accuracy. This was a clear improvement over the baseline and shows that the activation setup had a strong effect on model performance in this notebook.

Exp 4, which used the SGD optimizer with momentum, reached **57.60%**, so it performed worse than the baseline and was not the best result. Exp 5b, using batch size 32, reached **66.20%**, showing that batch size also affected training performance.

The animal-class subset reached **71.20%** in one run, which was also one of the strongest results. However, the later final animal-class output reached only **57.00%** and showed signs of overfitting. This suggests that the selected class subset and training configuration both had a large effect on the final accuracy.

7. AMD Cloud / ROCm comparison context

This AMD Cloud / ROCm walkthrough is useful as a comparison against Google Colab Free. Google Colab Free can be helpful for small experiments, but it often has changing GPU availability, runtime limits, and less predictable performance.

In this AMD Cloud notebook, the ROCm-enabled PyTorch environment allowed the CNN experiments to run with GPU support. This made it possible to test several architecture and

training changes in one workflow and compare their effects more comfortably.

The important comparison point is that the AMD Cloud / ROCm run produced a best observed test accuracy of **71.90%**, while also showing how sensitive the result was to optimizer choice, batch size, model structure, and class selection.

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import TensorDataset, DataLoader
import torchvision
import numpy as np
import matplotlib.pyplot as plt

print(f"PyTorch version: {torch.__version__}")
print(f"GPU available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")

PyTorch version: 2.10.0+cu128
GPU available: True
GPU: Tesla T4
```

Figure 11. Google Colab Free environment used for comparison, showing PyTorch with CUDA support and a Tesla T4 GPU.

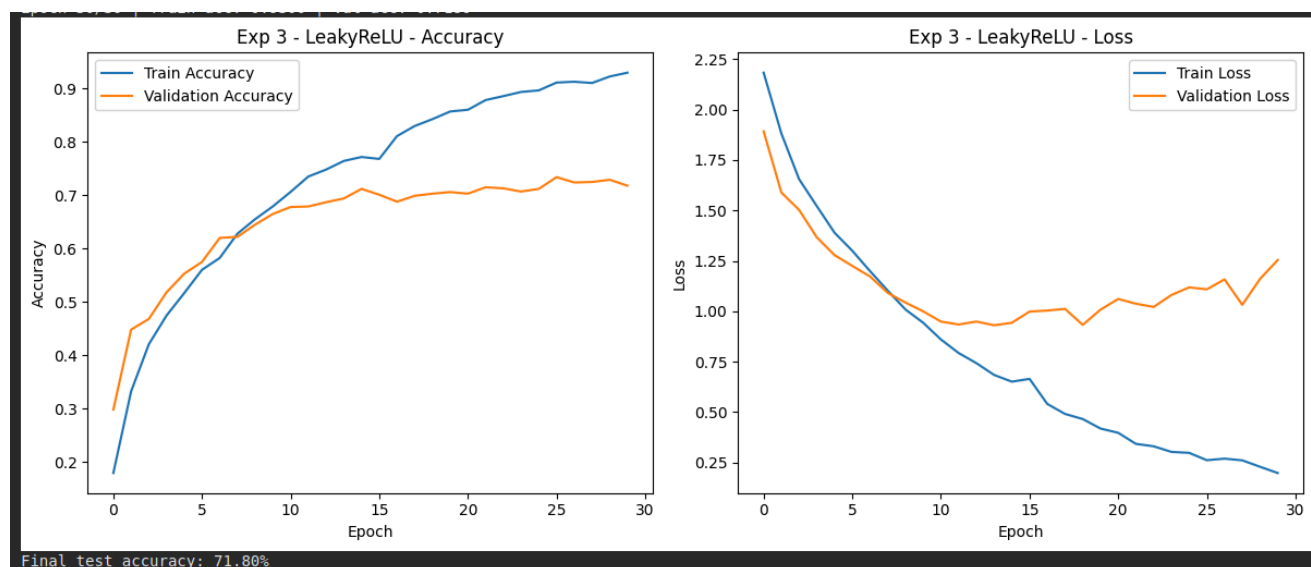


Figure 12. Google Colab Free result for Exp 3 using LeakyReLU, reaching **71.80%** final test accuracy.

8. Conclusion

The AMD Cloud / ROCm Jupyter notebook successfully trained several CNN models on selected CIFAR-100 subsets. The notebook confirmed that GPU acceleration was available through a ROCm-enabled PyTorch build, and the dataset was filtered to 10 classes with 5,000 training images and 1,000 test images.

The experiments produced final test accuracies ranging from **57.00%** to **71.90%**. The best result was **71.90%**, achieved by **Exp 3: LeakyReLU with Adam**, not by the SGD optimizer experiment. The animal-class subset also performed strongly in one run with **71.20%** final test accuracy, although the later final animal-class output reached **57.00%**.

Overall, the results show that CNN performance on CIFAR-100 depends strongly on training choices. Activation function, optimizer, batch size, architecture, and selected classes all affected the final accuracy. The AMD Cloud / ROCm run provides a useful comparison point for evaluating how the same type of CNN workflow behaves compared with Google Colab Free.